

Two failure modes in cell-based CCP estimation of dynamic discrete choice models

Aditya Maiti

ORCID 0009-0004-2501-1459

Independent research

Abstract. Conditional choice probability inversion is a standard way to estimate dynamic discrete choice models. One common implementation discretises a continuous state space into cells, estimates choice probabilities and transitions by counting within cells, and recovers flow utilities in closed form. This report documents two failure modes of that implementation. Missing values in the next-period state propagate through a percentile-based discretiser and collapse every next-period observation into a single cell, which makes the continuation value constant and removes the dynamics from the estimator. Additive smoothing in cells with no observed instance of the reference action makes the estimated value function a deterministic function of cell sample size. Both failures passed 21 unit tests, a spectral radius check, and a sensitivity sweep over the smoothing parameter. The report explains why each check passed, gives three diagnostics that detect the failures, and supplies a corrected implementation.

Version 1.0.0

Released 2026-10-07

DOI 10.5281/zenodo.23208936

Code github.com/ADITYA-WORK-MAITI/cell-based-ccp-failure-modes

Licence MIT for code, CC BY 4.0 for text and figures

This PDF is generated from REPORT.md by `rebuild/md2pdf.py`. The markdown file is authoritative. Every number is traceable through docs/TRACEABILITY.md.

Summary

Conditional choice probability inversion is a standard way to estimate dynamic discrete choice models. One common implementation discretises a continuous state space into cells, estimates choice probabilities and transitions by counting within cells, and then recovers flow utilities in closed form. This report documents two failure modes of that implementation. Both were found in a working research codebase. Both produced estimates that looked reasonable and passed every check the codebase contained.

The first failure removes the dynamics from the estimator. Missing values in the next-period state propagate through a percentile-based discretiser and send every next-period observation to a single cell. The transition matrix then has one reachable column. The continuation value becomes a constant, so the estimator is no longer solving a dynamic problem. In the case documented here, the continuation value for one action took exactly one distinct value across all 625 cells.

The second failure makes the estimated value function a function of cell sample size. When a cell contains no observed instances of the reference action, additive smoothing sets that action's probability to a ratio built from the cell count. The recovered value function in those cells then tracks the log of the cell count. In the case documented here, 432 of 625 cells had no observed instance of the reference action.

Both failures are easy to detect once stated. Neither was detected by 21 unit tests, by a spectral radius check, or by a sensitivity sweep over the smoothing parameter. The report explains why each of those checks passed, and gives three diagnostics that would have caught the failures.

A corrected implementation is included. On simulated data where the estimator is correctly specified, it recovers known utilities with root mean squared error falling from 0.4598 to 0.0548 as the number of agents grows from 500 to 32,000. The log-log slope is -0.4890 against a target near -0.5.

Who this is for, and what it covers

This report is for people who estimate dynamic discrete choice models or offline inverse reinforcement learning models using discretised states and counted transitions. The failures described are properties of that estimation strategy. They are not properties of the underlying theory, which is correct and well established.

The report does not contain a new theorem. It does not contain an empirical finding about any real population. It documents two implementation failures, gives verified evidence for each, and supplies code and diagnostics.

The project that produced these findings started as an attempt at a theoretical contribution. That attempt failed, for reasons given in the section on prior work. The failure modes are what survived.

The estimator

The setting is a stationary infinite-horizon Markov decision problem. An agent observes a state and chooses one of several actions. One action is terminal. Choosing it ends the problem and yields no further payoff. The other actions continue the problem and move the state according to a transition kernel.

The agent receives independent type-I extreme value shocks to the payoff of each action. This gives choice probabilities of multinomial logit form. Write the choice probability of action a in state x as $\sigma(a|x)$.

Utilities are identified only after one action's utility is fixed externally. The implementation studied here fixes the terminal action's utility at a measured value $w(x)$. With that fixed, the value function and the flow utilities follow in closed form:

$$\begin{aligned} V(x) &= w(x) - \ln \sigma(\theta|x) + \kappa \\ Y_a(x) &= \ln[\sigma(a|x) / \sigma(\theta|x)] + w(x) \\ u(x,a) &= Y_a(x) - \beta (P_a V)(x) \end{aligned}$$

Here β is the discount factor and κ is the mean of the choice shock. The term $(P_a V)(x)$ is the continuation value. It is the only place where the dynamics enter.

The implementation estimates σ and P_a by counting. It winsorises the state variables at their 1st and 99th percentiles, splits each dimension into quintiles, and assigns each observation to one of 625 cells. Choice probabilities come from action counts within cells. Transition matrices come from counts of cell-to-cell moves within cells.

Failure one: missing values collapse the transition matrix

The mechanism

Panel data on a terminal decision always has missing next-period states. Two sources are unavoidable. An agent that chooses the terminal action has no next period. An agent still active in the final period of the panel has no observed next period either.

The implementation stored those cases as rows of missing values. It then passed the whole array of next-period states to the same discretiser used for the current-period states. That discretiser computes bin edges with `numpy.percentile`.

`numpy.percentile` returns a missing value when any input element is missing. The function to use is `numpy.nanpercentile`. With the plain version, every bin edge became missing. Clipping to missing bounds made every value missing. The final step assigns bins with `numpy.searchsorted`, which drives every lookup against an all-missing edge array to the highest index. Every observation landed in the last cell.

The evidence

The failure was reproduced on the real panel that produced the project's published estimates. The panel has 146,985 rows. Of these, 2,970 had a fully missing next-period state.

Running the original code path on that panel gave the following. The winsorisation bounds were missing in every dimension. Every winsorised next-period value was missing. The set of distinct next-period cell indices was {624}. A single cell out of 625.

The stored estimation output confirms the consequence. For the first continuation action, the estimated transition matrix has one reachable column, which holds all of the mass. For the second, one column holds 99.52% of the mass. The remaining mass belongs to cells that had no observed transitions and received a uniform fallback.

The continuation value for the first continuation action took exactly one distinct value across all 625 cells. That value was 7.701230708570058.

So the estimator reduced to $u(x, a) = Y_a(x)$ minus a constant. The dynamic programme contributed nothing that varied with the state.

Why the existing checks passed

Three checks existed. All three passed.

The codebase ran 21 unit tests. None of them called the top-level estimation function. They tested the recovery formulas on hand-written inputs, and the transition estimator on random inputs that contained no missing values. The code path where the failure lived was never run by a test.

The codebase computed the spectral radius of βM , where M combines the choice probabilities and the transition matrices. The check requires this to be below one. With a degenerate transition matrix it still is. Measured on a deliberately degenerate matrix and on a healthy one, the value was 0.96624 in both cases. The check cannot distinguish them.

The symptoms in the simulation results were visible but were read as noise. Root mean squared error on recovered utilities was about 3.3 to 3.5 and did not fall as the sample grew. It rose as the panel got longer. Bootstrap coverage of utility levels was 0.2612 and 0.2319 against a nominal 0.95, while coverage of the difference between two continuation utilities was 0.9946. That split is the signature of an error common to both actions. It cancels in the difference and contaminates both levels.

Failure two: smoothing makes the value function track cell size

The mechanism

The recovery formula needs $\ln \sigma(\theta|x)$. When a cell contains no observed instances of the terminal action, the counted probability is zero and the logarithm is undefined. The standard fix is additive smoothing. With smoothing parameter α and A actions, the smoothed probability is

$$\sigma(\theta|k) = (n_{\{k, \theta\}} + \alpha) / (n_k + A \alpha)$$

In a cell with no observed terminal actions, $n_{\{k, \theta\}}$ is zero. The expression reduces to $\alpha / (n_k + A \alpha)$. Substituting into the recovery formula gives

$$V(k) = w(k) + \ln(n_k + A \alpha) - \ln \alpha + \kappa$$

The estimated value function in those cells is a deterministic function of the cell count. Nothing about the agents' behaviour in the cell enters it, because nothing was observed.

The evidence

The real panel had 432 of 625 cells with no observed terminal action. That is 69% of cells.

The project's own output listed the five cells with the largest estimated discrepancy and the five with the smallest, each with its reported choice probability and cell count. Solving the smoothing formula for the integer count of terminal actions reproduces every reported probability to better than 5 parts in ten million:

cell	reported $\sigma(0)$	cell count	implied terminal-action count
525	0.000048	2090	0
526	0.000071	1400	0
531	0.000078	1289	0
532	0.000090	1117	0
344	0.000117	855	0
179	0.174603	6	1
229	0.132530	8	1
78	0.074205	28	2
109	0.053040	77	4
84	0.049759	62	3

The five cells with the largest estimated discrepancy contain no observed terminal actions at all. The five with the smallest contain between one and four. The full reported range of the quantity of interest is spanned by cells that are either pure prior or driven by at most four events.

Why the sensitivity sweep could not detect it

The project ran a sensitivity sweep over the smoothing parameter. That sweep was the main defence on this point. It could not have worked.

Take a correctly specified simulation with a 0.1% rate of the terminal action. Look only at the cells with no observed terminal action. The correlation between the estimated value function and the log cell count is 0.505 there. It is 0.505 at every smoothing value tested: 0.01, 0.1, 0.5 and 1.0. The reason is in the formula. $V(k)$ picks up $\ln(n_k + A\alpha)$. Changing α barely moves that term.

Level error does respond to the smoothing parameter, and not monotonically. Root mean squared error on one recovered utility was 0.6400, 0.3717, 0.3910 and 0.4451 at the four values. There is an interior optimum near 0.1. But the structural dependence on cell size is untouched. A sweep that reports only error levels will show a sensible-looking optimum and reveal nothing about the mechanism.

The corrected estimator

Three changes fix the first failure.

Bin edges are computed once, from the current-period states, with `numpy.nanpercentile`.

The same edges are reused for the next-period states. In the original code the two periods were binned against separate percentile grids, so a transition matrix entry mapped between two different partitions of the state space. That is a second defect, independent of the missing-value problem.

Rows with missing values return an explicit sentinel instead of a cell index. They are then excluded from counting rather than absorbed into a cell.

Cells with no observed transitions for an action still receive a uniform fallback, but the number of such cells is now reported rather than left silent.

The second failure cannot be fixed by code. It is a property of having no data. What can be fixed is the reporting. The corrected implementation returns the count of cells with no observed terminal action, and the correlation between the estimated value function and the log cell count within those cells.

What it recovers

The corrected estimator was tested on a simulation built so that the estimator is exactly correctly specified. The simulation uses a finite state space. The terminal action's utility is set to exactly the value the estimator assumes. The rate of the terminal action is calibrated by shifting the level of the continuation utilities, which leaves that assumption exact.

This matters because the original simulation did not have this property. It set the terminal action's choice-specific value as a function of the true value function, which is not what the estimator assumes. So the original error measurements were measuring specification bias, not sampling error. That is why they did not fall as the sample grew.

With the corrected simulation and the corrected estimator, root mean squared error on recovered utilities falls as expected:

agents	observations	error on first utility	error on second	error on the difference
500	7,841	0.4598	0.3600	0.4744
2,000	30,210	0.2385	0.1218	0.2099
8,000	121,420	0.1603	0.1011	0.1107
32,000	482,592	0.0548	0.0505	0.0396

The log-log slope of the first column against the number of agents is -0.4890. The expected slope for this kind of estimator is -0.5.

Three checks worth running

The first check is a test that recovers known utilities end to end. Build a simulation in which the estimator's identifying assumption holds exactly. Run the full estimation function, not its parts. Assert that the error falls with sample size at the expected rate. This single test catches both failures described here, and it is the test the original codebase did not have.

The second check counts reachable columns in each estimated transition matrix. A healthy matrix over K cells reaches many columns. A degenerate one reaches one. The check is two lines and it is not implied by the spectral radius condition.

The third check applies to any estimate that depends on smoothing. Report two numbers

alongside it. The first is the count of cells with no observed instance of the reference action. The second is the correlation between the estimated value function and the log cell count within those cells. Both belong next to the estimates, not in an appendix.

Relation to prior work

The theory behind this estimator is settled. Rust (1987) gives the value function with extreme value shocks. Hotz and Miller (1993) give the inversion. Magnac and Thesmar (2002) give the identification result. They show these models are not identified and determine the exact degree of underidentification. Point identification requires setting three things: the distribution of the unobserved shocks, the discount rate, and current and future preferences in one reference alternative. The third is the anchor used here. Arcidiacono and Miller (2011) give the representation behind the closed-form recovery. Their Lemma 1 establishes a function of the choice probabilities equal to the gap between the ex-ante value and a conditional value function. Their Theorem 1 uses it to express a conditional value function in terms of choice probabilities a few periods ahead. Rust (1997) covers the computational cost of solving these models. He proves that randomised algorithms break the curse of dimensionality for a subclass of Markov decision problems called discrete decision processes.

Kang (2026) is a lecture note covering the intersection of dynamic discrete choice and entropy-regularised inverse reinforcement learning. It proves the equivalence of the two formulations, develops the Magnac and Thesmar identification result, and works through the classical computational approaches. Its section 3.2 is titled "The Anchor-Action Assumption", and it states that fixing one action's reward is a normalisation that fixes the location of the reward. It names a terminal action as the example.

This matters because the project that produced this report originally claimed a theoretical contribution about terminal actions and shock conventions. That claim does not survive comparison with Kang. The convention difference between the two formulations is a shift of the reward, the normalisation question is treated there directly, and the quantity the project built its argument around follows from material in that note. The theoretical work has been withdrawn.

Kang does treat the computational obstructions of grid discretisation, including the cost growing as h^{-d} in the state dimension. It does not treat the finite-sample behaviour of the counted estimates on such a grid. Searching its full text finds no mention of additive smoothing, of cells with no observations, of missing-value handling, or of degeneracy in an estimated transition matrix. The two failure modes in this report appear to be undocumented.

On the inverse reinforcement learning side, Kostrikov et al. (2019) show that imitation learning methods commonly assign zero reward to absorbing states, often implicitly, and that this biases the recovered policy. Their remedy is to learn the absorbing-state reward rather than assume it. That concerns the specification of the terminal state. It is a different question from the estimation failures here.

Limitations

The evidence for both failures comes from one codebase and one panel. The mechanisms are general, because they follow from the formulas and from standard library behaviour. The frequencies are not. How often cells come up empty depends on the grid, the sample size and the rate of the terminal action.

The corrected estimator is tested only on simulations with a finite state space, where it is exactly correctly specified. That is the right first test, because it isolates sampling error. It says nothing about discretisation bias when the true state space is continuous. That bias does not vanish as the sample grows and is a separate question.

The report makes no empirical claim about firms or any other population. The panel that exposed these failures has problems of its own, described in the accompanying audit, and no estimate from it is reported here as a finding.

The second failure has no remedy inside the estimator. Reporting it is not the same as fixing it. When most cells have no observed instance of the reference action, the right conclusion is that the utility levels are not estimable on that grid at that sample size.

Code and reproduction

The implementation is in `rebuild/`. It requires Python, numpy and matplotlib.

```
python rebuild/tests.py      # 9 tests
python rebuild/run_all.py    # all numbers and figures
```

`rebuild/ccp.py` holds the estimator, the forward solver and the simulation.

`rebuild/tests.py` holds the tests, including one that reproduces the first failure on purpose and confirms the fix. `rebuild/run_all.py` regenerates every number in this report and writes them to `rebuild/out/results.json`.

The original code, results and figures are kept unchanged in the repository. They are invalid for the reasons given above and are listed in `WITHDRAWN_OUTPUTS.md`. They are kept because they are the evidence for this report.

A note on how these failures survived

The codebase was audited several times before these failures were found. Those audits checked that every citation matched its source, down to the page and equation number. They were careful and they were useful. They also did not ask whether the continuation value varied across states.

The checks that existed were aimed at the parts of the work that were easiest to check. The failure was in the part that was load-bearing. That gap is the most transferable thing in this report.

Method and tooling

This work was done with AI assistance, under the author's direction, with the author responsible for every claim in it.

The reason for stating that plainly is the subject of this report. It is about results that passed their own checks and were wrong anyway. A reader is entitled to know how the work was produced before deciding what weight to give it.

Nothing here rests on trusting that record. Every mathematical statement has a check in `rebuild/verify_math.py`, every reported number has a named source in `docs/TRACEABILITY.md`, and the estimator has nine tests in `rebuild/tests.py`. Run them.

References

Each entry was checked on 4 October 2026 against the PDF held in references/, the arXiv record, or the publisher record. The source of the check is given in brackets.

Arcidiacono, P. and Miller, R. A. (2011). Conditional Choice Probability Estimation of Dynamic Discrete Choice Models With Unobserved Heterogeneity. *Econometrica* 79(6), 1823-1867. doi:10.3982/ECTA7743. [Publisher record. The copy in references/ is the April 2011 working paper version.]

Cao, H., Cohen, S. N. and Szpruch, L. (2021). Identifiability in inverse reinforcement learning. arXiv:2106.03498. [arXiv record and PDF front matter.]

Haarnoja, T., Tang, H., Abbeel, P. and Levine, S. (2017). Reinforcement Learning with Deep Energy-Based Policies. *Proceedings of the 34th International Conference on Machine Learning*, PMLR 70. arXiv:1702.08165. [arXiv record, and the venue line on page 1 of the PDF.]

Hotz, V. J. and Miller, R. A. (1993). Conditional Choice Probabilities and the Estimation of Dynamic Models. *The Review of Economic Studies* 60(3), 497-529. doi:10.2307/2298122. [PDF front matter and publisher record.]

Kang, E. H. (2026). A Lecture Note on Offline RL and IRL, Part II: Foundations of Inverse Reinforcement Learning and Dynamic Discrete Choice Models. arXiv:2605.30843. [arXiv record and full text. Read in full for this report.]

Kostrikov, I., Agrawal, K. K., Dwibedi, D., Levine, S. and Thompson, J. (2019). Discriminator-Actor-Critic: Addressing Sample Inefficiency and Reward Bias in Adversarial Imitation Learning. *International Conference on Learning Representations*. arXiv:1809.02925. [arXiv record and full text. The absorbing-state claim is stated in Sections 4.1 and 4.2.]

Magnac, T. and Thesmar, D. (2002). Identifying Dynamic Discrete Decision Processes. *Econometrica* 70(2), 801-816. doi:10.1111/1468-0262.00306. [PDF front matter and publisher record.]

Rust, J. (1987). Optimal Replacement of GMC Bus Engines: An Empirical Model of Harold Zurcher. *Econometrica* 55(5), 999-1033. doi:10.2307/1911259. [Publisher record. The copy in references/ is a scan with no machine-readable text, so the content attribution was checked instead against Arcidiacono and Miller (2011) Section 2.2, which states the log-sum-exp form of the conditional value function and attributes it to Rust (1987).]

Rust, J. (1997). Using Randomization to Break the Curse of Dimensionality. *Econometrica* 65(3), 487-516. doi:10.2307/2171751. [PDF front matter and publisher record.]

Train, K. (2009). Logit. Chapter 3 of *Discrete Choice Methods with Simulation*, 2nd edition, pages 38-79. Cambridge University Press. doi:10.1017/CBO9780511753930.004. [Publisher

record and PDF chapter heading. Cited for the history of the logit derivation.]

Ziebart, B. D., Maas, A., Bagnell, J. A. and Dey, A. K. (2008). Maximum Entropy Inverse Reinforcement Learning. *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence*, pages 1433-1438. No DOI was assigned by the publisher, and a Crossref search on 5 October 2026 returned none. [PDF front matter, and the running header on page 2 for the venue and pagination.]